

Team Involvement in Demonstration

Date	29-09-2026
Team ID	SWTID-2026-6684
Project Name	ComicCraft - AI Comic Story Creator using Gemini Models
Maximum Marks	1 Mark

Team Involvement in Demonstration:

This document records the active participation and roles of each team member during the project demonstration. It ensures that every member contributes meaningfully to the presentation and that responsibilities are distributed fairly and clearly.

S.No	Team Member Name	Role in Demo	Section Presented	Contribution Summary	Participation (Active / Passive)
1	Lead Developer	Presenter & Demo Lead	System Architecture & Gemini Integration	Demonstrated prompt engineering and API setup for story & panel generation using Gemini API.	Active
2	UI/UX Developer	UI Presenter	Frontend UI & Layout Engine	Showcased the Streamlit UI, comic grid layout rendering, and export features.	Active
3	Backend Engineer	Technical Presenter	Backend Logic & Data Flow	Explained state management, error handling, and structured output formatting.	Active
4	QA / Documentation	Support Presenter	Testing & Documentation	Presented user workflow, test cases, and project documentation.	Active
5	-	-	-	-	-
6	-	-	-	-	-

Team Coordination Notes:

Team Leader / Coordinator	Lead Developer
Overall Team Coordination Rating (1-5)	5 / 5
Any issues during demo	

	None. All live model generation and UI demonstrations executed smoothly within allotted time.
How issues were resolved	Implemented local state caching and fallback options in case of API rate limits.

Project Implementation Code (Gemini Model Integration):

```
import google.generativeai as genai
import json

# Configure Gemini API Key
genai.configure(api_key="YOUR_GEMINI_API_KEY")

def generate_comic_story(prompt):
    model = genai.GenerativeModel('gemini-2.5-flash')

    structured_prompt = f"""
    Create a 4-panel comic strip based on the concept: "{prompt}".
    Return response STRICTLY in JSON format with keys:
    - "title": Comic Title
    - "panels": List of 4 objects, each containing:
        - "panel_number": int
        - "visual_description": detailed scene description for image generation
        - "dialogue": character dialogue or caption
    """

    response = model.generate_content(
        structured_prompt,
        generation_config={"response_mime_type": "application/json"}
    )

    return json.loads(response.text)

# Example Usage
if __name__ == "__main__":
    concept = "A futuristic robot discovering a flower in a cyberpunk city"
    comic_data = generate_comic_story(concept)
    print(json.dumps(comic_data, indent=2))
```